

Projeto em Java Spring Boot no IntelliJ para captura de dados jurídicos no site da AASP, tradução com NLP para termos não técnicos de fácil entendimento para leigos, arquivamento dos dados originais e traduzidos em perfil individual.

Estrutura de pacotes:

```
.com.tg.meu_processo
├── controller
├── dto
├── entity
│   └── enums
├── repository
├── security
├── service
└── MeuProcessoApplication
```

```
resources
└── application.properties
```

Arquivos de código dentro de cada pacote

```
controller
    AuthController
    MovimentacaoController
    ProcessoController
    UsuarioController
```

```
dto
    MovimentacaoCreateDto
    MovimentacaoDto
    ProcessoCreateDto
    ProcessoDto
    RecuperarSenhaDto
    SenhaUpdateDto
    UsuarioCreateDto
    UsuarioDto
    LoginDto
```

entity

enums

PerfilUsuario

StatusProcesso

Movimentacao

Processo

Usuario

repository

MovimentacaoRepository

ProcessoRepository

UsuarioRepository

security

AuthenticatedUserService

JwtAuthenticatedFilter

JwtService

SecurityConfig

UserDetailsServiceImpl

service

MovimentacaoService

UsuarioService

ProcessoService

```
package com.tg.meu_processo.controller;

import com.tg.meu_processo.dto.LoginDTO;
import com.tg.meu_processo.entity.Usuario;
import com.tg.meu_processo.repository.UsuarioRepository;
import com.tg.meu_processo.security.JwtService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import java.util.Map;

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final UsuarioRepository usuarioRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginDTO dto) {
        Usuario usuario = usuarioRepository.findByEmail(dto.email())
            .orElse(null);

        if (usuario == null || !passwordEncoder.matches(dto.senha(), usuario.getSenha())) {
            return ResponseEntity.status(401).body(Map.of("erro", "Credenciais inválidas"));
        }

        String token = jwtService.generateToken(usuario.getEmail(), usuario.getPerfil().name(),
            usuario.getId());

        return ResponseEntity.ok(Map.of(
            "token", token,
            "perfil", usuario.getPerfil(),
            "nome", usuario.getNome(),
            "id", usuario.getId()
        ));
    }
}
```

```
package com.tg.meu_processo.controller;
```

```
import com.tg.meu_processo.dto.MovimentacaoCreateDTO;  
import com.tg.meu_processo.dto.MovimentacaoDTO;  
import com.tg.meu_processo.service.MovimentacaoService;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
@RestController
```

```
@RequestMapping("/api/movimentacoes")
```

```
public class MovimentacaoController {
```

```
    private final MovimentacaoService service;
```

```
    public MovimentacaoController(MovimentacaoService service) {  
        this.service = service;  
    }
```

```
    @GetMapping("/processo/{processoid}")  
    public List<MovimentacaoDTO> listarPorProcesso(@PathVariable Long processoid) {  
        return service.listarPorProcesso(processoid);  
    }
```

```
    @GetMapping("/{id}")  
    public MovimentacaoDTO buscar(@PathVariable Long id) {  
        return service.buscarPorId(id);  
    }
```

```
    @PostMapping  
    public MovimentacaoDTO criar(@RequestBody MovimentacaoCreateDTO dto) {  
        return service.criar(dto);  
    }
```

```
    @DeleteMapping("/{id}")  
    public ResponseEntity<Void> deletar(@PathVariable Long id) {  
        service.deletar(id);  
        return ResponseEntity.noContent().build();  
    }
```

```
}
```

```
package com.tg.meu_processo.controller;
```

```
import com.tg.meu_processo.dto.ProcessoCreatedTO;  
import com.tg.meu_processo.dto.ProcessoDTO;  
import com.tg.meu_processo.security.AuthenticatedUserService;  
import com.tg.meu_processo.service.ProcessoService;  
import lombok.RequiredArgsConstructor;  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.security.access.prepost.PreAuthorize;  
import org.springframework.web.bind.annotation.*;  
import java.util.List;
```

```
@RestController
```

```
@RequestMapping("/api/processos")
```

```
@RequiredArgsConstructor
```

```
public class ProcessoController {
```

```
    private final ProcessoService service;
```

```
    private final AuthenticatedUserService authService;
```

```
    @GetMapping
```

```
    public ResponseEntity<List<ProcessoDTO>> listar() {
```

```
        if (authService.isAdmin()) {
```

```
            return ResponseEntity.ok(service.listarTodos());
```

```
        } else if (authService.isAdvogado()) {
```

```
            return ResponseEntity.ok(service.listarPorAdvogado(authService.getUsuariId()));
```

```
        } else if (authService.isCliente()) {
```

```
            return ResponseEntity.ok(service.listarPorCliente(authService.getUsuariId()));
```

```
        }
```

```
        return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
```

```
    }
```

```
    @GetMapping("/{id}")
```

```
    public ResponseEntity<ProcessoDTO> buscarPorId(@PathVariable Long id) {
```

```
        ProcessoDTO processo = service.buscarPorId(id);
```

ProcessoController

```
    if (authService.isAdmin()) {
        return ResponseEntity.ok(processo);
    }
    if (authService.isAdvogado() && processo.advogadold().equals(authService.getUsuariold())) {
        return ResponseEntity.ok(processo);
    }
    if (authService.isCliente() && processo.clienteld().equals(authService.getUsuariold())) {
        return ResponseEntity.ok(processo);
    }
    return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
}

@PostMapping
@PreAuthorize("hasRole('ADMINISTRADOR')")
public ResponseEntity<ProcessoDTO> criar(@RequestBody ProcessoCreatedDTO dto) {
    return ResponseEntity.status(HttpStatus.CREATED).body(service.criar(dto));
}

@PutMapping("/{id}")
@PreAuthorize("hasRole('ADMINISTRADOR')")
public ResponseEntity<ProcessoDTO> atualizar(@PathVariable Long id, @RequestBody
ProcessoCreatedDTO dto) {
    return ResponseEntity.ok(service.atualizar(id, dto));
}

@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMINISTRADOR')")
public ResponseEntity<String> deletar(@PathVariable Long id) {
    if (!service.existePorId(id)) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Processo não encontrado");
    }
    service.deletar(id);
    return ResponseEntity.ok("Processo deletado com sucesso");
}
}
```

```
@PostMapping
@PreAuthorize("hasAnyRole('ADMINISTRADOR', 'ADVOGADO')")
public ProcessoDTO criar(@RequestBody ProcessoCreateDTO dto) {
    return service.criar(dto);
}

@PutMapping("/{id}")
@PreAuthorize("hasAnyRole('ADMINISTRADOR', 'ADVOGADO')")
public ProcessoDTO atualizar(@PathVariable Long id, @RequestBody ProcessoCreateDTO dto)
{
    return service.atualizar(id, dto);
}

@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMINISTRADOR')")
public ResponseEntity<Void> deletar(@PathVariable Long id) {
    service.deletar(id);
    return ResponseEntity.noContent().build();
}

private boolean podeAcessar(ProcessoDTO p) {
    if (authenticatedUserService.isAdmin()) return true;
    if (authenticatedUserService.isAdvogado()) return
p.advogadId().equals(authenticatedUserService.getUsuariId());
    if (authenticatedUserService.isCliente()) return
p.clientId().equals(authenticatedUserService.getUsuariId());
    return false;
}
}
```

```
package com.tg.meu_processo.controller;

import com.tg.meu_processo.dto.SenhaUpdateDTO;
import com.tg.meu_processo.dto.UsuarioCreatedDTO;
import com.tg.meu_processo.dto.UsuarioDTO;
import com.tg.meu_processo.security.AuthenticatedUserService;
import com.tg.meu_processo.service.UsuarioService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Map;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/usuarios")
@RequiredArgsConstructor
public class UsuarioController {

    private final UsuarioService service;
    private final AuthenticatedUserService authService;

    @GetMapping
    @PreAuthorize("hasRole('ADMINISTRADOR')")
    public ResponseEntity<List<UsuarioDTO>> listarTodos() {
        return ResponseEntity.ok(service.listarTodos());
    }

    @GetMapping("/{id}")
    public ResponseEntity<UsuarioDTO> buscarPorId(@PathVariable Long id) {
        if (authService.isAdmin() || authService.isOwner(id)) {
            return ResponseEntity.ok(service.buscarPorId(id));
        }
        return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
    }

    @PostMapping
    @PreAuthorize("hasRole('ADMINISTRADOR')")
    public ResponseEntity<Map<String, Object>> criar(@Valid @RequestBody
        UsuarioCreateDTO dto)
    {
        return ResponseEntity.status(HttpStatus.CREATED).body(service.criarPorAdmin(dto));
    }
}
```


UsuarioController

```
@PutMapping("/{id}")
public ResponseEntity<UsuarioDTO> atualizar(@PathVariable Long id, @RequestBody
UsuarioCreatedDTO dto) {
    if (authService.isAdmin() || authService.isOwner(id)) {
        return ResponseEntity.ok(service.atualizar(id, dto));
    }
    return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
}

@PutMapping("/{id}/senha")
public ResponseEntity<String> alterarSenha(@PathVariable Long id, @RequestBody
SenhaUpdateDTO dto) {
    if (authService.isOwner(id)) {
        service.alterarSenha(id, dto);
        return ResponseEntity.ok("Senha alterada com sucesso");
    }
    return ResponseEntity.status(HttpStatus.FORBIDDEN).body("Acesso negado");
}

@PostMapping("/recuperar-senha")
public ResponseEntity<Map<String, String>> recuperarSenha(@RequestBody Map<String,
String> body) {
    return ResponseEntity.ok(service.recuperarSenha(body.get("email")));
}

@DeleteMapping("/{id}")
public ResponseEntity<String> deletar(@PathVariable Long id) {
    if (authService.isAdmin() || authService.isOwner(id)) {
        if (!service.existePorId(id)) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Usuário não
encontrado");
        }
        service.deletar(id);
        return ResponseEntity.ok("Usuário deletado com sucesso");
    }
    return ResponseEntity.status(HttpStatus.FORBIDDEN).body("Acesso negado");
}
}
```

RecuperarSenhaDTO

```
package com.tg.meu_processo.dto;
```

```
public record RecuperarSenhaDTO(  
    String email  
) {}
```

UsuarioCreateDTO

```
package com.tg.meu_processo.dto;
```

```
import  
com.tg.meu_processo.entity.enums.PerfilUsuario;  
import jakarta.validation.constraints.NotBlank;
```

```
public record UsuarioCreateDTO(  
    @NotBlank(message = "Nome é obrigatório")  
    String nome,  
  
    @NotBlank(message = "CPF é obrigatório")  
    String cpf,  
  
    String cargo,  
    String oab,  
  
    @NotBlank(message = "Email é obrigatório")  
    String email,  
  
    String whatsapp,  
    PerfilUsuario perfil  
) {}
```

MovimentacaoCreateDTO

```
package com.tg.meu_processo.dto;
```

```
import java.time.LocalDateTime;
```

```
public record MovimentacaoCreateDTO(  
    Long processold,  
    String descricaoOriginal,  
    LocalDateTime dataMovimentacao  
) {}
```

SenhaUpdateDTO

```
package com.tg.meu_processo.dto;
```

```
public record SenhaUpdateDTO(  
    String senhaAtual,  
    String novaSenha  
) {}
```

UsuarioDTO

```
package com.tg.meu_processo.dto;
```

```
import  
com.tg.meu_processo.entity.enums.  
PerfilUsuario;
```

```
public record UsuarioDTO(  
    Long id,  
    String nome,  
    String cpf,  
    String cargo,  
    String oab,  
    String email,  
    String whatsapp,  
    PerfilUsuario perfil  
) {}
```

MovimentacaoDTO

```
package com.tg.meu_processo.dto;
```

```
import java.time.LocalDateTime;
```

```
public record MovimentacaoDTO(  
    Long id,  
    Long processold,  
    String descricaoOriginal,  
    String descricaoTraduzida,  
    LocalDateTime dataMovimentacao,  
    Boolean notificacaoEnviada  
) {}
```

ProcessoCreateDTO

```
package com.tg.meu_processo.dto;

import com.tg.meu_processo.entity.enums.StatusProcesso;

public record ProcessoCreateDTO(
    String numeroProcesso,
    String titulo,
    String descricao,
    StatusProcesso status,
    String vara,
    String comarca,
    Long advogadold,
    Long clienteld
) {}
```

ProcessoDTO

```
package com.tg.meu_processo.dto;

import
com.tg.meu_processo.entity.enums.StatusProcesso;

public record ProcessoDTO(
    Long id,
    String numeroProcesso,
    String titulo,
    String descricao,
    StatusProcesso status,
    String vara,
    //String comarca,
    Long advogadold,
    String advogadoNome,
    Long clienteld,
    String clienteNome
) {}
```

LoginDTO

```
package com.tg.meu_processo.dto;

public record LoginDTO(String email, String senha) {}
```

```
package com.tg.meu_processo.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "movimentacoes")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Movimentacao {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(columnDefinition = "TEXT", nullable = false)
    private String descricaoOriginal;

    @Column(columnDefinition = "TEXT")
    private String descricaoTraduzida;

    @Column(nullable = false)
    private LocalDateTime dataMovimentacao;

    @ManyToOne
    @JoinColumn(name = "processo_id", nullable = false)
    private Processo processo;

    private Boolean notificacaoEnviada;

    @Column(nullable = false, updatable = false)
    private LocalDateTime dataCriacao;

    @PrePersist
    protected void onCreate() {
        dataCriacao = LocalDateTime.now();
        if (notificacaoEnviada == null) notificacaoEnviada = false;
    }
}
```

```
package com.tg.meu_processo.entity;
```

Processo.java

```
import com.tg.meu_processo.entity.enums.StatusProcesso;  
import jakarta.persistence.*;  
import lombok.*;  
import java.time.LocalDateTime;  
import java.util.List;
```

```
@Entity
```

```
@Table(name = "processos")
```

```
@Data
```

```
@NoArgsConstructor
```

```
@AllArgsConstructor
```

```
@Builder
```

```
public class Processo {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    @Column(nullable = false, unique = true, length = 30)
```

```
    private String numeroProcesso;
```

```
    @Column(length = 255)
```

```
    private String titulo;
```

```
    @Column(columnDefinition = "TEXT")
```

```
    private String descricao;
```

```
    @Column(length = 100)
```

```
    private String vara;
```

```
    // @Column(length = 100)
```

```
    // private String comarca;
```

```
    @Enumerated(EnumType.STRING)
```

```
    @Column(nullable = false)
```

```
    private StatusProcesso status;
```

```
    @ManyToOne
```

```
    @JoinColumn(name = "cliente_id", nullable = false)
```

```
    private Usuario cliente;
```

```
@ManyToOne
@JoinColumn(name = "advogado_id", nullable = false)
private Usuario advogado;

@OneToMany(mappedBy = "processo", cascade = CascadeType.ALL)
private List<Movimentacao> movimentacoes;

@Column(nullable = false, updatable = false)
private LocalDateTime dataCriacao;

private LocalDateTime dataAtualizacao;

@PrePersist
protected void onCreate() {
    dataCriacao = LocalDateTime.now();
    if (status == null) status = StatusProcesso.EM_ANDAMENTO;
}

@PreUpdate
protected void onUpdate() {
    dataAtualizacao = LocalDateTime.now();
}
}
```

```

package com.tg.meu_processo.entity;

import com.tg.meu_processo.entity.enums.PerfilUsuario;
import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "usuarios")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nome;

    @Column(nullable = false, unique = true, length = 14)
    private String cpf;

    @Column(length = 50)
    private String cargo; // apenas ADMIN

    @Column(length = 20)
    private String oab; // apenas ADVOGADO

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(length = 20)
    private String whatsapp;

    @Column(nullable = false)
    private String senha;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private PerfilUsuario perfil;

```

```
@Column(nullable = false, updatable = false)
private LocalDateTime dataCriacao;

private LocalDateTime dataAtualizacao;

@PrePersist
protected void onCreate() {
    dataCriacao = LocalDateTime.now();
}

@PreUpdate
protected void onUpdate() {
    dataAtualizacao = LocalDateTime.now();
}
}
```

enums

PerfilUsuario

```
package com.tg.meu_processo.entity.enums;

public enum PerfilUsuario {
    ADMINISTRADOR,
    ADVOGADO,
    CLIENTE
}
```

StatusProcesso

```
package com.tg.meu_processo.entity.enums;

public enum StatusProcesso {
    EM_ANDAMENTO,
    SUSPENSO,
    ARQUIVADO,
    ENCERRADO,
    AGUARDANDO_RECURSO
}
```


ProcessoRepository

```
package com.tg.meu_processo.repository;

import com.tg.meu_processo.entity.Processo;
import com.tg.meu_processo.entity.enums.StatusProcesso;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;

public interface ProcessoRepository extends JpaRepository<Processo, Long> {
    List<Processo> findByClienteld(Long clienteld);
    List<Processo> findByAdvogadold(Long advogadold);
    List<Processo> findByStatus(StatusProcesso status);
}
```

UsuarioRepository

```
package com.tg.meu_processo.repository;

import com.tg.meu_processo.entity.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByEmail(String email);
    boolean existsByEmail(String email);
    boolean existsByCpf(String cpf);
}
```

MovimentacaoRepository

```
package com.tg.meu_processo.repository;

import com.tg.meu_processo.entity.Movimentacao;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface MovimentacaoRepository extends JpaRepository<Movimentacao, Long> {
    List<Movimentacao> findByProcessoldOrderByDataMovimentacaoDesc(Long processold);
    List<Movimentacao> findByNotificacaoEnviadaFalse();
}
```

```
//serviço de contexto do usuário  
package com.tg.meu_processo.security;
```

AuthenticatedUserService

```
import com.tg.meu_processo.entity.Usuario;  
import com.tg.meu_processo.entity.enums.PerfilUsuario;  
import com.tg.meu_processo.repository.UsuarioRepository;  
import lombok.RequiredArgsConstructor;  
import org.springframework.security.core.context.SecurityContextHolder;  
import org.springframework.stereotype.Service;  
  
@Service  
@RequiredArgsConstructor  
public class AuthenticatedUserService {  
  
    private final UsuarioRepository usuarioRepository;  
  
    public Usuario getUsuarioLogado() {  
        String email = SecurityContextHolder.getContext().getAuthentication().getName();  
        return usuarioRepository.findByEmail(email).orElse(null);  
    }  
  
    public boolean isAdmin() {  
        Usuario u = getUsuarioLogado();  
        return u != null && u.getPerfil() == PerfilUsuario.ADMINISTRADOR;  
    }  
  
    public boolean isAdvogado() {  
        Usuario u = getUsuarioLogado();  
        return u != null && u.getPerfil() == PerfilUsuario.ADVOGADO;  
    }  
  
    public boolean isCliente() {  
        Usuario u = getUsuarioLogado();  
        return u != null && u.getPerfil() == PerfilUsuario.CLIENTE;  
    }  
  
    public boolean isOwner(Long userId) {  
        Usuario u = getUsuarioLogado();  
        return u != null && u.getId().equals(userId);  
    }  
  
    public Long getUsuarioid() {  
        Usuario u = getUsuarioLogado();  
        return u != null ? u.getId() : null;  
    }  
}
```

```
package com.tg.meu_processo.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsServiceImpl userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");
        System.out.println("=== JWT FILTER ===");
        System.out.println("Auth Header: " + authHeader);

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            System.out.println("Token ausente ou inválido");
            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);
        String email = jwtService.extractEmail(token);
        System.out.println("Email extraído: " + email);

        /*if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }
    }
}
```

```
String token = authHeader.substring(7);
String email = jwtService.extractEmail(token);
*/

if (email != null && SecurityContextHolder.getContext().getAuthentication() == null) {
    UserDetails userDetails = userDetailsService.loadUserByUsername(email);

    if (jwtService.isTokenValid(token)) {
        UsernamePasswordAuthenticationToken authToken = new
            UsernamePasswordAuthenticationToken(
                userDetails, null, userDetails.getAuthorities()
            );
        authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(authToken);
    }
}

filterChain.doFilter(request, response);
}
}
```

```
package com.tg.meu_processo.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.security.Key;
import java.util.Date;

@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private Long expiration;

    private Key getSigningKey() {
        return Keys.hmacShaKeyFor(secret.getBytes());
    }

    public String generateToken(String email, String perfil, Long userId) {
        return Jwts.builder()
            .setSubject(email)
            .claim("perfil", perfil)
            .claim("userId", userId)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expiration))
            .signWith(getSigningKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    public Claims extractClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
    }
}
```

```
public String extractEmail(String token) {  
    return extractClaims(token).getSubject();  
}  
  
public boolean isTokenValid(String token) {  
    try {  
        return extractClaims(token).getExpiration().after(new Date());  
    } catch (Exception e) {  
        return false;  
    }  
}  
}
```

SecurityConfig

```
package com.tg.meu_processo.security;

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.PUT, "/api/usuarios/*/senha").authenticated()
                .requestMatchers("/api/usuarios/recuperar-senha").permitAll()
                .requestMatchers(HttpMethod.OPTIONS, "/*").permitAll()
                .anyRequest().authenticated()
            )
    }
}
```

SecurityConfig

```
.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

return http.build();
}

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(Arrays.asList("http://localhost:5173", "http://localhost:3000"));
    config.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(Arrays.asList("*"));
    config.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
Exception {
    return config.getAuthenticationManager();
}
}
```



```
package com.tg.meu_processo.security;

import com.tg.meu_processo.entity.Usuario;
import com.tg.meu_processo.repository.UsuarioRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Collections;

@Service
@RequiredArgsConstructor
public class UserDetailsServiceImpl implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        Usuario usuario = usuarioRepository.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException("Usuário não encontrado"));

        return new User(
            usuario.getEmail(),
            usuario.getSenha(),
            Collections.singletonList(new SimpleGrantedAuthority("ROLE_" + usuario.getPerfil().name()))
        );
    }
}
```

```
package com.tg.meu_processo.service;

import com.tg.meu_processo.dto.MovimentacaoCreateDTO;
import com.tg.meu_processo.dto.MovimentacaoDTO;
import com.tg.meu_processo.entity.Movimentacao;
import com.tg.meu_processo.repository.MovimentacaoRepository;
import com.tg.meu_processo.repository.ProcessoRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class MovimentacaoService {

    private final MovimentacaoRepository repository;
    private final ProcessoRepository processoRepository;

    public MovimentacaoService(MovimentacaoRepository repository, ProcessoRepository processoRepository) {
        this.repository = repository;
        this.processoRepository = processoRepository;
    }

    public List<MovimentacaoDTO> listarPorProcesso(Long processold) {
        return repository.findByProcessoldOrderByDataMovimentacaoDesc(processold).stream()
            .map(this::toDTO)
            .toList();
    }

    public MovimentacaoDTO buscarPorId(Long id) {
        return repository.findById(id)
            .map(this::toDTO)
            .orElseThrow(() -> new RuntimeException("Movimentação não encontrada"));
    }

    public MovimentacaoDTO criar(MovimentacaoCreateDTO dto) {
        Movimentacao mov = new Movimentacao();
        mov.setDescricaoOriginal(dto.descricaoOriginal());
        mov.setDataMovimentacao(dto.dataMovimentacao());
        mov.setNotificacaoEnviada(false);
        mov.setProcesso(processoRepository.findById(dto.processold())
            .orElseThrow(() -> new RuntimeException("Processo não encontrado")));
        return toDTO(repository.save(mov));
    }
}
```

MovimentacaoService

```
public void deletar(Long id) {  
    repository.deleteById(id);  
}  
  
private MovimentacaoDTO toDTO(Movimentacao m) {  
    return new MovimentacaoDTO(  
        m.getId(), m.getProcesso().getId(),  
        m.getDescricaoOriginal(), m.getDescricaoTraduzida(),  
        m.getDataMovimentacao(), m.getNotificacaoEnviada()  
    );  
}  
}
```

```
package com.tg.meu_processo.service;

import com.tg.meu_processo.dto.ProcessoCreateDTO;
import com.tg.meu_processo.dto.ProcessoDTO;
import com.tg.meu_processo.entity.Processo;
import com.tg.meu_processo.entity.Usuario;
import com.tg.meu_processo.entity.enums.PerfilUsuario;
import com.tg.meu_processo.repository.ProcessoRepository;
import com.tg.meu_processo.repository.UsuarioRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
@RequiredArgsConstructor
public class ProcessoService {

    private final ProcessoRepository repository;
    private final UsuarioRepository usuarioRepository;

    public boolean existePorId(Long id) { return repository.existsById(id);}

    public List<ProcessoDTO> listarTodos() {
        return repository.findAll().stream()
            .map(this::toDTO)
            .toList();
    }

    public List<ProcessoDTO> listarPorAdvogado(Long advogadoid) {
        return repository.findByAdvogadoid(advogadoid).stream()
            .map(this::toDTO)
            .toList();
    }

    public List<ProcessoDTO> listarPorCliente(Long clienteid) {
        return repository.findByClienteid(clienteid).stream()
            .map(this::toDTO)
            .toList();
    }

    public ProcessoDTO buscarPorId(Long id) {
        return repository.findById(id)
            .map(this::toDTO)
            .orElseThrow(() -> new RuntimeException("Processo não encontrado"));
    }
}
```

```
public ProcessoDTO criar(ProcessoCreateDTO dto) {
    Usuario advogado = usuarioRepository.findById(dto.advogadId())
        .filter(u -> u.getPerfil() == PerfilUsuario.ADVOGADO)
        .orElseThrow(() -> new RuntimeException("Advogado não encontrado"));

    Usuario cliente = usuarioRepository.findById(dto.clientId())
        .filter(u -> u.getPerfil() == PerfilUsuario.CLIENTE)
        .orElseThrow(() -> new RuntimeException("Cliente não encontrado"));

    Processo processo = new Processo();
    processo.setNumeroProcesso(dto.numeroProcesso());
    processo.setTitulo(dto.titulo());
    processo.setDescricao(dto.descricao());
    processo.setStatus(dto.status());
    processo.setVara(dto.vara());
    processo.setAdvogado(advogado);
    processo.setCliente(cliente);

    return toDTO(repository.save(processo));
}
```

```
public ProcessoDTO atualizar(Long id, ProcessoCreateDTO dto) {
    Processo processo = repository.findById(id)
        .orElseThrow(() -> new RuntimeException("Processo não encontrado"));

    processo.setNumeroProcesso(dto.numeroProcesso());
    processo.setTitulo(dto.titulo());
    processo.setDescricao(dto.descricao());
    processo.setStatus(dto.status());
    processo.setVara(dto.vara());

    return toDTO(repository.save(processo));
}
```

```
public void deletar(Long id) {
    repository.deleteById(id);
}
```

```
private ProcessoDTO toDTO(Processo p) {
    return new ProcessoDTO(
        p.getId(), p.getNumeroProcesso(),
        p.getTitulo(), p.getDescricao(),
        p.getStatus(), p.getVara(),
        p.getAdvogado().getId(), p.getAdvogado().getNome(),
        p.getCliente().getId(), p.getCliente().getNome()
    );
}
}
```

```
package com.tg.meu_processo.service;

import com.tg.meu_processo.dto.*;
import com.tg.meu_processo.entity.Usuario;
import com.tg.meu_processo.entity.enums.PerfilUsuario;
import com.tg.meu_processo.repository.UsuarioRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.security.SecureRandom;
import java.util.List;
import java.util.Map;

@Service
@RequiredArgsConstructor
public class UsuarioService {

    private final UsuarioRepository repository;
    private final PasswordEncoder passwordEncoder;

    public boolean existePorId(Long id) {
        return repository.existsById(id);
    }

    public List<UsuarioDTO> listarTodos() {
        return repository.findAll().stream()
            .map(this::toDTO)
            .toList();
    }

    public UsuarioDTO buscarPorId(Long id) {
        return repository.findById(id)
            .map(this::toDTO)
            .orElseThrow(() -> new RuntimeException("Usuário não encontrado"));
    }

    public Map<String, Object> criarPorAdmin(UsuarioCreateDTO dto) {
        if (repository.existsByEmail(dto.email())) {
            throw new RuntimeException("Email já cadastrado");
        }
        if (repository.existsByCpf(dto.cpf())) {
            throw new RuntimeException("CPF já cadastrado");
        }
        String senhaGerada = gerarSenha4Digitos();
```

```
Usuario usuario = Usuario.builder()
    .nome(dto.nome())
    .cpf(dto.cpf())
    .cargo(dto.perfil() == PerfilUsuario.ADMINISTRADOR ? dto.cargo() : null)
    .oab(dto.perfil() == PerfilUsuario.ADVOGADO ? dto.oab() : null)
    .email(dto.email())
    .whatsapp(dto.whatsapp())
    .senha(passwordEncoder.encode(senhaGerada))
    .perfil(dto.perfil())
    .build();
```

```
Usuario saved = repository.save(usuario);
```

```
return Map.of(
    "id", saved.getId(),
    "email", saved.getEmail(),
    "senhaTemporaria", senhaGerada,
    "mensagem", "Usuário criado. Envie a senha ao usuário."
);
}
```

```
public UsuarioDTO atualizar(Long id, UsuarioCreateDTO dto) {
    Usuario usuario = repository.findById(id)
        .orElseThrow(() -> new RuntimeException("Usuário não encontrado"));

    usuario.setNome(dto.nome());
    usuario.setCpf(dto.cpf());
    usuario.setEmail(dto.email());
    usuario.setWhatsapp(dto.whatsapp());

    if (usuario.getPerfil() == PerfilUsuario.ADMINISTRADOR) {
        usuario.setCargo(dto.cargo());
    }
    if (usuario.getPerfil() == PerfilUsuario.ADVOGADO) {
        usuario.setOab(dto.oab());
    }

    return toDTO(repository.save(usuario));
}
```

UsuarioService

```
public void alterarSenha(Long userId, SenhaUpdateDTO dto) {
    Usuario usuario = repository.findById(userId)
        .orElseThrow(() -> new RuntimeException("Usuário não encontrado"));

    if (!passwordEncoder.matches(dto.senhaAtual(), usuario.getSenha())) {
        throw new RuntimeException("Senha atual incorreta");
    }

    usuario.setSenha(passwordEncoder.encode(dto.novaSenha()));
    repository.save(usuario);
}

public Map<String, String> recuperarSenha(String email) {
    Usuario usuario = repository.findByEmail(email)
        .orElseThrow(() -> new RuntimeException("Email não encontrado"));

    String novaSenha = gerarSenha4Digitos();
    usuario.setSenha(passwordEncoder.encode(novaSenha));
    repository.save(usuario);

    // TODO: Enviar por email/whatsapp
    return Map.of(
        "mensagem", "Nova senha gerada",
        "novaSenha", novaSenha // Em produção, enviar por email, não retornar!
    );
}

public void deletar(Long id) {
    repository.deleteById(id);
}

private String gerarSenha4Digitos() {
    SecureRandom random = new SecureRandom();
    int numero = random.nextInt(10000);
    return String.format("%04d", numero);
}

private UsuarioDTO toDTO(Usuario u) {
    return new UsuarioDTO(
        u.getId(), u.getNome(), u.getCpf(), u.getCargo(),
        u.getOab(), u.getEmail(), u.getWhatsapp(), u.getPerfil()
    );
}
```


UsuarioService

```
public boolean isOwner(Long id) {  
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();  
  
    if (auth == null || auth.getPrincipal() == null) {  
        System.out.println("Auth is null");  
        return false;  
    }  
    Usuario usuario = (Usuario) auth.getPrincipal();  
  
    System.out.println("ID do token: " + usuario.getId() + " | ID da URL: " + id);  
    return usuario.getId().equals(id);  
}
```

MeuProcessoApplication

```
package com.tg.meu_processo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MeuProcessoApplication {

    public static void main(String[] args) {
        SpringApplication.run(MeuProcessoApplication.class, args);
    }

}
```

application.properties

===== BANCO POSTGRESQL =====

spring.datasource.url=jdbc:postgresql://localhost:5432/tg_legaldb
spring.datasource.username=postgres
spring.datasource.password=minhasenha
spring.datasource.driver-class-name=org.postgresql.Driver

===== JPA/HIBERNATE =====

spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

===== SERVIDOR =====

server.port=8080

===== SEGURANÇA (desabilitar temporariamente para testes iniciais) =====

spring.security.user.name=admin
spring.security.user.password=admin123
jwt.secret=minha-chave-secreta-super-segura-com-pelo-menos-256-bits-ok
jwt.expiration=86400000

===== LOGS =====

logging.level.com.tg.meu_processo=DEBUG
logging.level.org.springframework.security=DEBUG
spring.application.name=meu_processo

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.12-SNAPSHOT</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.tg</groupId>
  <artifactId>meu_processo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>meu_processo</name>
  <description>Sistema de acompanhamento de processos</description>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>

</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <annotationProcessorPaths>
          <path>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
          </path>
        </annotationProcessorPaths>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludes>
          <exclude>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
          </exclude>
        </excludes>
      </configuration>
    </plugin>
  </plugins>
</build>

<repositories>
  <repository>
    <id>spring-snapshots</id>
    <name>Spring Snapshots</name>
    <url>https://repo.spring.io/snapshot</url>
    <releases>
      <enabled>false</enabled>
    </releases>
  </repository>
</repositories>
```

```
<pluginRepositories>
  <pluginRepository>
    <id>spring-snapshots</id>
    <name>Spring Snapshots</name>
    <url>https://repo.spring.io/snapshot</url>
    <releases>
      <enabled>false</enabled>
    </releases>
  </pluginRepository>
</pluginRepositories>
```

```
</project>
```

version: '3.8'

services:

db:

image: postgres:15

environment:

POSTGRES_DB: tg_legaldb

POSTGRES_USER: postgres

POSTGRES_PASSWORD: J0403366#

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

volumes:

postgres_data:

//Admin criado automaticamente

DataInitializer

```
//cria admin automaticamente
package com.tg.meu_processo;

import com.tg.meu_processo.entity.Usuario;
import com.tg.meu_processo.entity.enums.PerfilUsuario;
import com.tg.meu_processo.repository.UsuarioRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
public class DataInitializer implements CommandLineRunner {

    private final UsuarioRepository usuarioRepository;
    private final PasswordEncoder passwordEncoder;

    @Override
    public void run(String... args) {
        if (usuarioRepository.findByEmail("admin@sistema.com").isEmpty()) {
            Usuario admin = Usuario.builder()
                .nome("Administrador")
                .cpf("000000000000")
                .email("admin@sistema.com")
                .senha(passwordEncoder.encode("admin123"))
                .perfil(PerfilUsuario.ADMINISTRADOR)
                .build();
            usuarioRepository.save(admin);
            System.out.println(">>> Admin criado: admin@sistema.com / admin123");
        }
    }
}
```

Usuários cadastrados

```
{
  "id": 1,
  "nome": "Administrador",
  "cpf": "00000000000",
  "cargo": null,
  "oab": null,
  "email": "admin@sistema.com",
  "whatsapp": null,
  "perfil": "ADMINISTRADOR"
},
{
  "id": 2,
  "nome": "João Silva",
  "cpf": "12345678901",
  "cargo": null,
  "oab": null,
  "email": "joao@cliente.com",
  "whatsapp": "16999999999",
  "perfil": "CLIENTE"
},
{
  "id": 3,
  "nome": "Maria Souza",
  "cpf": "98765432100",
  "cargo": null,
  "oab": "SP123456",
  "email": "maria@advogado.com",
  "whatsapp": "16988888888",
  "perfil": "ADVOGADO"
}
```

Login Admin

POST http://localhost:8080/api/auth/login

Content-Type: application/json

```
{  
  "email": "admin@sistema.com",  
  "senha": "admin123"  
}
```

Criar Cliente (usar token do login)

POST http://localhost:8080/api/usuarios

Content-Type: application/json

Authorization: Bearer COLE_O_TOKEN_AQUI

```
{  
  "nome": "João Silva",  
  "cpf": "12345678901",  
  "email": "joao@cliente.com",  
  "whatsapp": "16999999999",  
  "perfil": "CLIENTE"  
}
```

Criar Advogado

POST http://localhost:8080/api/usuarios

Content-Type: application/json

Authorization: Bearer COLE_O_TOKEN_AQUI

```
{  
  "nome": "Maria Souza",  
  "cpf": "98765432100",  
  "oab": "SP123456",  
  "email": "maria@advogado.com",  
  "whatsapp": "16988888888",  
  "perfil": "ADVOGADO"  
}
```

Listar Usuários

GET http://localhost:8080/api/usuarios

Authorization: Bearer COLE_O_TOKEN_AQUI

Buscar Usuário por ID

GET http://localhost:8080/api/usuarios/1

Authorization: Bearer COLE_O_TOKEN_AQUI

Atualizar Usuário

PUT http://localhost:8080/api/usuarios/1

Content-Type: application/json

Authorization: Bearer COLE_O_TOKEN_AQUI

```
"nome": "João Silva Atualizado",  
"cpf": "12345678901",  
"email": "joao@cliente.com",  
"whatsapp": "16999999999",  
"perfil": "CLIENTE"
```

Deletar Usuário

```
DELETE http://localhost:8080/api/usuarios/id  
Authorization: Bearer COLE_O_TOKEN_AQUI
```

Criar Processo

```
POST http://localhost:8080/api/processos  
Content-Type: application/json  
Authorization: Bearer COLE_O_TOKEN_AQUI
```

```
"numeroProcesso": "0001234-56.2026.8.26.0196",  
"titulo": "Ação Trabalhista",  
"descricao": "Desvio de função",  
"status": "EM_ANDAMENTO",  
"vara": "1ª Vara",  
"advogadold": 3,  
"clienteld": 2
```

Listar Processos

```
GET http://localhost:8080/api/processos  
Authorization: Bearer COLE_O_TOKEN_AQUI
```

Buscar Processo por ID

```
GET http://localhost:8080/api/processos/id  
Authorization: Bearer COLE_O_TOKEN_AQUI
```

Atualizar Processo

```
PUT http://localhost:8080/api/processos/id  
Content-Type: application/json  
Authorization: Bearer COLE_O_TOKEN_AQUI
```

```
"numeroProcesso": "0001234-56.2026.8.26.0196",  
"titulo": "Ação Trabalhista Atualizado",  
"descricao": "Desvio de função",  
"status": "EM_ANDAMENTO",  
"vara": "1ª Vara",  
"advogadold": 3,  
"clienteld": 4
```

Deletar Processo

```
DELETE http://localhost:8080/api/processos/id  
Authorization: Bearer COLE_O_TOKEN_AQUI
```

Alterar senha

PUT http://localhost:8080/api/usuarios/{id}/senha

Content-Type: application/jsonAuthorization: Bearer COLE_O_TOKEN_AQUI

```
{  
  "senhaAtual": "****",  
  "novaSenha": "****"  
}
```

```
{  
  "mensagem": "Usuário criado. Envie a senha ao usuário.",  
  "id": 2,  
  "email": "joao@cliente.com",  
  "senhaTemporaria": "1069"  
}
```

```
{  
  "mensagem": "Usuário criado. Envie a senha ao usuário.",  
  "id": 3,  
  "email": "maria@advogado.com",  
  "senhaTemporaria": "8001"  
}
```

```
{  
  "mensagem": "Usuário criado. Envie a senha ao usuário.",  
  "id": 4,  
  "email": "joao2@cliente.com",  
  "senhaTemporaria": "5808" (atual = cli123)  
} (deletado)
```

```
{  
  "mensagem": "Usuário criado. Envie a senha ao usuário.",  
  "id": 5,  
  "email": "drajus@advogado.com",  
  "senhaTemporaria": "2707"  
}
```